2

Pandas Basics

In this chapter, you will see a brief introduction to the Pandas series and Dataframes, which are two basic data structures for storing data in Pandas. Next, you will see how to create these data structures and some basic functions that you can perform with Pandas. You will then study how to import datasets into a Pandas dataframe using various input sources. Finally, the chapter concludes with an explanation of the techniques for handling missing data in Pandas dataframes.

Pandas comes installed with default Python installation. You can also install Pandas via the following PIP command:

```
pip install pandas
```

2.1. Pandas Series

A Pandas series is a data structure that stores data in the form of a column. A series is normally used to store information about a particular attribute in your dataset. Let's see how you can create a series in Pandas.

2.1.1. Creating Pandas Series

There are different ways to create a series with Pandas. The following script imports the Pandas module and then calls the Series() class constructor to create an empty series. Here is how to do that:

Script 1:

```
# empty series
```

```
import pandas as pd

my_series = pd.Series()
print (my_series)
```

You can also create a series using a NumPy array. But, first, you need to pass the array to the Series() class constructor, as shown in the script below.

Script 2:

```
# series using numpy array

import pandas as pd
import numpy as np

my_array = np.array([ 10 , 20 , 30 , 40 , 50 ])

my_series = pd.Series(my_array)
print (my_series)
```

Output:

```
0 10
1 20
2 30
3 40
4 50
dtype: int32
```

In the above output, you can see that the indexes for a series start from 0 to 1 less than the number of items in the series. You can also define custom indexes for your series. To do so, you need to pass your list of indexes to the index attribute of the Series class, as shown in the script below:

Script 3:

```
# series with custom indexes

import pandas as pd
import numpy as np

my_array = np.array([ 10 , 20 , 30 , 40 , 50 ])
```

```
my_series = pd.Series(my_array, index = ["num1", "num2", "num3", "num4", "num5"])
print (my_series)
```

Output:

```
num1 10
num2 20
num3 30
num4 40
num5 50
dtype: int32
```

You can also create a series by directly passing a Python list to the Series() class constructor.

Script 4:

```
# series using a list

import pandas as pd
import numpy as np

my_series = pd.Series([ 10 , 20 , 30 , 40 , 50 ], index = ["num1", "num2", "num3", "num4", "num5"])
print (my_series)
```

Output:

```
num1 10
num2 20
num3 30
num4 40
num5 50
dtype: int64
```

Finally, a scaler value can also be used to define a series. In case you pass a list of indexes, the scaler value will be repeated the number of times equal to the items in the index list. Here is an example:

Script 5:

```
# series using a scaler

import pandas as pd
import numpy as np

my_series = pd.Series( 25 , index = ["num1", "num2", "num3", "num4", "num5"])
print (my_series)
```

Output:

```
num1 25
num2 25
num3 25
num4 25
num5 25
dtype: int64
```

Finally, you can also create a series using a dictionary. In this case, the dictionary keys will become series indexes while the dictionary values are inserted as series items. Here is an example:

Script 6:

```
# series using dictionary

my_dict = {'num1' : 6 ,
'num2' : 7 ,
'num3' : 8 }

my_series = pd.Series(my_dict)
print (my_series)
```

Output:

```
num1 6
num2 7
num3 8
dtype: int64
```

7.1.2. Useful Operations on Pandas Series

Let's see some of the useful operations you can perform with the Pandas series.

You can use square brackets as well as index labels to access series items, as shown in the following script:

Script 7:

```
## Accessing Items

import pandas as pd
my_series = pd.Series([ 10 , 20 , 30 , 40 , 50 ], index = ["num1", "num2", "num3", "num4", "num5"])
print (my_series[ 0 ])
print (my_series['num3'])
```

Output:

```
10
30
```

Using the min() and max() functions from the NumPy module, you can find the maximum and minimum values, respectively, from a series. Look at the following script for reference.

Script 8:

```
## Finding Maximum and Minimum Values

import pandas as pd
import numpy as np

my_series = pd.Series([ 5 , 8 , 2 , 11 , 9 ])

print (np.min(my_series))
print (np.max(my_series))
```

Output:

```
2
11
```

Similarly, the mean() method from the NumPy module can find the mean of a Pandas series, as shown in the following script.

Script 9:

```
## Finding Mean

import pandas as pd
import numpy as np

my_series = pd.Series([ 5 , 8 , 2 , 11 , 9 ])

print (my_series.mean())
```

Output:

```
7.0
```

The following script finds the median value of a Pandas series.

Script 10:

```
## Finding Median

import pandas as pd
import numpy as np

my_series = pd.Series([ 5 , 8 , 2 , 11 , 9 ])

print (my_series.median())
```

Output:

```
8.0
```

You can also find the data type of a Pandas series using the dtype attribute. Here is an example:

Script 11:

```
## Finding Data Type

import pandas as pd
import numpy as np

my_series = pd.Series([ 5 , 8 , 2 , 11 , 9 ])

print (my_series.dtype)
```

Output:

```
int64
```

A Pandas series can also be converted to a Python list using the tolist() method, as shown in the script below:

Script 12:

```
## Converting to List

import pandas as pd
import numpy as np

my_series = pd.Series([ 5 , 8 , 2 , 11 , 9 ])

print (my_series.tolist())
```

Output:

```
[5, 8, 2, 11, 9]
```

2.2. Pandas Dataframe

A Pandas dataframe is a tabular data structure that stores data in the form of rows and columns. As a standard, the rows correspond to records while columns refer to attributes. In simplest words, a Pandas dataframe is a collection of series.

2.2.1. Creating a Pandas Dataframe

As is the case with a series, there are multiple ways to create a Pandas dataframe.

To create an empty dataframe, you can use the DataFrame class from the Pandas module, as shown below:

Script 13:

```
# empty pandas dataframe

import pandas as pd

my_df = pd.DataFrame() print (my_df)
```

Output:

```
EmptyDataFrame
Columns: []
Index: []
```

You can create a Pandas dataframe using a list of lists. Each sublist in the outer list corresponds to a row in a dataframe. Each item within a sublist becomes an attribute value.

To specify column headers, you need to pass a list of values to the columns attribute of DataFrame class.

Here is an example of how you can create a Pandas dataframe using a list.

Script 14:

```
# dataframe using list of lists

import pandas as pd

scores = [['Mathematics', 85 ], ['English', 91 ], ['History', 95 ]]

my_df = pd.DataFrame(scores, columns = ['Subject', 'Score'])
```

```
my_df
```

Output:

	Subject	Score
0	Mathematics	85
1	English	91
2	History	95

Similarly, you can create a Pandas dataframe using a dictionary. One of the ways is to create a dictionary where keys correspond to column headers. In contrast, corresponding dictionary values are a list, which corresponds to the column values in the Pandas dataframe.

Here is an example for your reference:

Script 15:

```
# dataframe using dictionaries

import pandas as pd

scores = {'Subject':["Mathematics", "History", "English", "Science", "Arts"],
'Score':[ 98 , 75 , 68 , 82 , 99 ]
    }

my_df = pd.DataFrame(scores)
my_df
```

Output:

	Subject	Score
0	Mathematics	98
1	History	75
2	English	68
3	Science	82
4	Arts	99

Another way to create a Pandas dataframe is using a list of dictionaries. Each dictionary corresponds to one row. Here is an example of how to do that.

Script 16:

```
# dataframe using list of dictionaries

import pandas as pd

scores = [
    {'Subject':'Mathematics', 'Score': 85 },
    {'Subject':'History', 'Score': 98 },
    {'Subject':'English', 'Score': 76 },
    {'Subject':'Science', 'Score': 72 },
    {'Subject':'Arts', 'Score': 95 },
    ]

my_df = pd.DataFrame(scores)
my_df
```

Output:

	Subject	Score
0	Mathematics	85
1	History	98
2	English	76
3	Science	72
4	Arts	95

The dictionaries within the list used to create a Pandas dataframe need not be of the same size.

For example, in the script below, the fourth dictionary in the list contains only one item, unlike the rest of the dictionaries in this list. The corresponding dataframe will contain a null value in place of the second item, as shown in the output of the script below:

Script 17:

```
# dataframe using list of dictionaries
# with null items

import pandas as pd

scores = [
    {'Subject':'Mathematics', 'Score': 85 },
    {'Subject':'History', 'Score': 98 },
    {'Subject':'English', 'Score': 76 },
    {'Score': 72 },
    {'Subject':'Arts', 'Score': 95 },
    ]

my_df = pd.DataFrame(scores)
my_df
```

Output:

	Subject	Score
0	Mathematics	85
1	History	98
2	English	76
3	NaN	72
4	Arts	95

2.2.2. Basic Operations on Pandas Dataframe

Let's now see some of the basic operations that you can perform on Pandas dataframes.

To view the top(N) rows of a dataframe, you can call the head() method, as shown in the script below:

Script 18:

```
# viewing header

import pandas as pd

scores = [
    {'Subject':'Mathematics', 'Score': 85 },
    {'Subject':'History', 'Score': 98 },
    {'Subject':'English', 'Score': 76 },
    {'Subject':'Science', 'Score': 72 },
    {'Subject':'Arts', 'Score': 95 },
    ]

my_df = pd.DataFrame(scores)
my_df.head( 2 )
```

Output:

	Subject	Score
0	Mathematics	85
1	History	98

To view the last N rows, you can use the tail() method. Here is an example:

Script 19:

```
# viewing tail

my_df = pd.DataFrame(scores)

my_df.tail( 2 )
```

Output:

	Subject	Score
3	Science	72
4	Arts	95

You can also get a summary of your Pandas dataframe using the info() method.

Script 20:

```
# gettingdataframe info

my_df = pd.DataFrame(scores)

my_df.info()
```

In the output below, you can see the number of entries in your Pandas dataframe, the number of columns along with their column type, and so on.

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 2 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   Subject  5 non-null      object
 1   Score    5 non-null      int64
dtypes: int64(1), object(1)
memory usage: 208.0+ bytes
```

Finally, to get information such as mean, minimum, maximum, standard deviation, etc., for numeric columns in your Pandas dataframe, you can use the describe() method, as shown in the script below:

Script 21:

```
# getting info about numeric columns

my_df = pd.DataFrame(scores)
```

```
my_df.describe()
```

Output:

	Score
count	5.000000
mean	85.200000
std	11.388591
min	72.000000
25%	76.000000
50%	85.000000
75%	95.000000
max	98.000000

2.3. Importing Data in Pandas

You can import data from various sources into your Pandas dataframe. Some of them are discussed in this section.

2.3.1. Importing CSV Files

A CSV file is a type of file where each line contains a single record, and all the columns are separated from each other via a comma.

You can read CSV files using the **read_csv()** function of the Pandas dataframe, as shown below. The "iris_data.csv" file is available in the *Data* folder of the book resources.

Script 22:

```
import pandas as pd
titanic_data = pd.read_csv(r"D:\Datasets\iris_data.csv")
titanic_data.head()
```

If you print the dataframe header, you should see that the header contains five columns that contain different information about iris plants.

Output:

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

In some cases, CSV files do not contain any header. In such cases, the read_csv() method treats the first row of the CSV file as the dataframe header.

To specify custom headers for your CSV files, you need to pass the list of headers to the names attribute of the read_ csv() method, as shown in the script below. You can find the "pima-indians-diabetes.csv" file in the *Data* folder of the book resources.

Script 23:

```
headers = ["Preg", "Glucose", "BP", "skinThick", "Insulin", "BMI", "DPF", "Age", "Class"]

patient_data = pd.read_csv(r"D:\Datasets\pima-indians- diabetes.csv", names = headers)

patient_data.head()
```

In the output below, you can see the custom headers that you passed in the list to the read_csv() method's name attribute.

Output:

	Preg	Glucose	BP	skinThick	Insulin	BMI	DPF	Age	Class
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

2.3.2. Importing TSV Files

TSV files are similar to CSV files. But in a TSV file, the delimiter used to separate columns is a single tab. The **read_csv()** function can be used to read a TSV file. However, you have to pass "\t" as a value for the "sep" attribute, as shown below.

Note: You can find the "iris_data.tsv" file in the *Data* folder of the book resources.

Script 24:

```
import pandas as pd
patients_csv = pd.read_csv(r"D:\Datasets\iris_data.tsv", sep=' \t ')
patients_csv.head()
```

Output:

	Preg	Glucose	BP	skinThick	Insulin	BMI	DPF	Age	Class
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

2.3.3. Importing Data from Databases

Oftentimes, you need to import data from different databases into your Pandas dataframe. In this section, you will see how to import data from various databases into a Python application.

Importing Data from SQL Server

To import data from Microsoft's SQL Server database, you need to first install the "pyodbc" module for Python. To do so, execute the following command on your command terminal.

```
$ pip install pyodbc
```

Next, you need to create a connection with your SQL server database. The **connect()** method of the "pyodbc" module can be used to create a connection. You have to pass the driver name, the server name, and the database name to the connect() method, as shown below.

Note: To run the following script, you need to create a database named *Titanic* with a table named *records* table. Explaining how to create a database and tables is beyond the scope of this book. You find further details at the link below.

Further Readings – CRUD Operations with SQL Server

To see how to create databases and tables with SQL Server, take a look at this link: https://bit.ly/2XwEgAV

In the following script, we connect to the Titanic database.

Script 25:

```
import pandas as pd
import pyodbc

sql_conn = pyodbc.connect('DRIVER={ODBC Driver 17 for SQL Server};
    SERVER=HOARE\SQLEXPRESS; DATABASE=Titanic; Trusted_Connection=yes')
```

Once the connection is established, you have to write an SQL SELECT query that fetches the desired record. The following SQL select query fetches all records from a *records* table.

In a Pandas dataframe, the query and the connection object are passed to the **pd_read_sql()** function to store the records returned by the query.

Finally, the dataframe header is printed to display the first five rows of the imported table.

Script 26:

```
query = "SELECT * FROM records;"
titanic_data = pd.read_sql(query, sql_conn)
titanic_data.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	False	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.250000	None	S
1	2	True	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.283302	C85	C
2	3	True	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.925000	None	S
3	4	True	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.099998	C123	S
4	5	False	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.050000	None	S

Importing Data from PostgreSQL

To import data from PostgreSQL, you will need to download the *SQLAlchemy* module. Execute the following pip statement to do so.

```
$ pip install SQLAlchemy
```

Next, you need to create an engine, which serves as a connection between the PostgreSQL server and the Python application. The following script shows how to create a connection engine. You need to replace your server and database name in the following script.

Script 27:

```
from sqlalchemy import create_engine
```

```
engine = create_engine('postgresql://postgres:abc123@
    localhost:5432/Titanic')
```

To store the records returned by the query in a Pandas dataframe, the query and the connection object are passed to the **pd_read_sql()** function of the Pandas dataframe. Finally, the dataframe header is printed to display the first five rows of the imported table.

Script 28:

```
import pandas as pd
titanic_data =pd.read_sql_query('select * from "records"',con=engine)
titanic_data.head()
```

Output:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	False	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.250000	None	S
1	2	True	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.283302	C85	C
2	3	True	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.925000	None	S
3	4	True	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.099998	C123	S
4	5	False	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.050000	None	S

Further Readings – CRUD Operations with PostgreSQL

To see how to create databases and tables with PostgreSQL, take a look at this link: https://bit.ly/2XyJr3f

Importing Data from SQLite

To import data from an SQLite database, you do not need any external module. You can use the default *sqlite3* module.

The first step is to connect to an SQLite database. To do so, you can use the **connect()** method of the sqlite3 module, as shown below.

Script 29:

```
import sqlite3
import pandas as pd
# Create your connection.
cnx = sqlite3.connect('E:/Titanic.db')
```

Next, you can call the pd_**read_sql()** function of the Pandas dataframe and pass it to the SELECT query and the database connection. Finally, the dataframe header is printed to display the first five rows of the imported table.

Script 30:

```
titanic_data = pd.read_sql_query("SELECT * FROM records", cnx)
titanic_data.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	False	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.250000	None	S
1	2	True	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.283302	C85	C
2	3	True	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.925000	None	S
3	4	True	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.099998	C123	S
4	5	False	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.050000	None	S

Further Readings – CRUD Operations with SQLite

To see how to create databases and tables with SQLite, take a look at this link: https://bit.ly/2BAXZXL

2.4. Handling Missing Values in Pandas

Missing values, as the name suggests, are those observations in the dataset that doesn't contain any value. Missing values can change the data patterns, and, therefore, it is extremely important to understand why missing values occur in the dataset and how to handle them.

In this section, you will see the different techniques with examples to handle missing values in your Pandas dataframes.

2.4.1. Handling Missing Numerical Values

To handle missing numerical data, we can use statistical techniques. The use of statistical techniques or algorithms to replace missing values with statistically generated values is called imputation.

In this section, you will see how to do median and mean imputation. Mean or median imputation is one of the most commonly used imputation techniques for handling missing numerical data.

In mean or median imputation, missing values in a column are replaced by the mean or median of all the remaining values in that particular column.

For instance, if you have a column with the following data:

Age
15
NA
20
25
40

In the above Age column, the second value is missing. Therefore, with mean and median imputation, you can replace the second value with either the mean or median of all the other values in the column. For instance, the following column contains the mean of all the remaining values, i.e., 25 in the second row. You could also replace this value with the median if you want.

Age
15
25
20
25
40

Let's see a practical example of mean and median imputation. First, we will import the Titanic dataset and find the columns that contain missing values. Then, we will apply mean and median imputation to the columns containing missing values. Finally, we will see the effect of applying mean and median imputation to the missing values.

You do not need to download the Titanic dataset. If you import the Seaborn library, the Titanic data will be downloaded with it. The following script imports the Titanic dataset and displays its first five rows.

Script 31:

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.rcParams["figure.figsize"] = [ 8 , 6 ]
sns.set_style("darkgrid")

titanic_data = sns.load_dataset('titanic')

titanic_data.head()
```

Output:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True

Let's filter some of the numeric columns from the dataset and see if they contain any missing values.

Script 32:

```
titanic_data = titanic_data[["survived", "pclass", "age", "fare"]]
titanic_data.head()
```

Output:

	survived	pclass	age	fare
0	0	3	22.0	7.2500
1	1	1	38.0	71.2833
2	1	3	26.0	7.9250
3	1	1	35.0	53.1000
4	0	3	35.0	8.0500

To find missing values from the aforementioned columns, you need to first call the **isnull()** method on the **titanic_data** dataframe, and then you need to call the **mean()** method, as shown below.

Script 33:

```
titanic_data.isnull().mean()
```

Output:

```
survived 0.000000
```

```
pclass 0.000000
age 0.198653
fare 0.000000
dtype: float64
```

The above output shows that only the *age* column contains missing values. And the ratio of missing values is around 19.86 percent.

Let's now find out the median and mean values for all the non- missing values in the *age* column.

Script 34:

```
median = titanic_data.age.median()
print (median)

mean = titanic_data.age.mean()
print (mean)
```

Output:

```
28.0
29.69911764705882
```

The age column has a median value of 28 and a mean value of 29.6991.

To plot the kernel density plots for the actual age and median and mean age, we will add columns to the Pandas dataframe.

Script 35:

```
import numpy as np

titanic_data['Median_Age'] = titanic_data.age.fillna(median)

titanic_data['Mean_Age'] = titanic_data.age.fillna(mean)

titanic_data['Mean_Age'] = np.round(titanic_data['Mean_Age'], 1 )

titanic_data.head( 20 )
```

The above script adds **Median_Age** and **Mean_Age** columns to the **titanic_data** dataframe and prints the first 20 records. Here is the output of the above script:

Output:

	survived	pclass	age	fare	Median_Age	Mean_Age
0	0	3	22.0	7.2500	22.0	22.0
1	1	1	38.0	71.2833	38.0	38.0
2	1	3	26.0	7.9250	26.0	26.0
3	1	1	35.0	53.1000	35.0	35.0
4	0	3	35.0	8.0500	35.0	35.0
5	0	3	NaN	8.4583	28.0	29.7
6	0	1	54.0	51.8625	54.0	54.0
7	0	3	2.0	21.0750	2.0	2.0
8	1	3	27.0	11.1333	27.0	27.0
9	1	2	14.0	30.0708	14.0	14.0
10	1	3	4.0	16.7000	4.0	4.0
11	1	1	58.0	26.5500	58.0	58.0
12	0	3	20.0	8.0500	20.0	20.0
13	0	3	39.0	31.2750	39.0	39.0
14	0	3	14.0	7.8542	14.0	14.0
15	1	2	55.0	16.0000	55.0	55.0
16	0	3	2.0	29.1250	2.0	2.0
17	1	2	NaN	13.0000	28.0	29.7
18	0	3	31.0	18.0000	31.0	31.0
19	1	3	NaN	7.2250	28.0	29.7

The highlighted rows in the above output show that NaN, i.e., null values in the **age** column, have been replaced by the median values in the **Median_Age** column and by mean values in the **Mean_Age** column.

The mean and median imputation can affect the data distribution for the columns containing the missing values. Specifically, the variance of the column is decreased by mean and median imputation now since more values are added to the center of the distribution. The following script plots the distribution of data for the **age** , **Median_Age** , and **Mean_Age** columns.

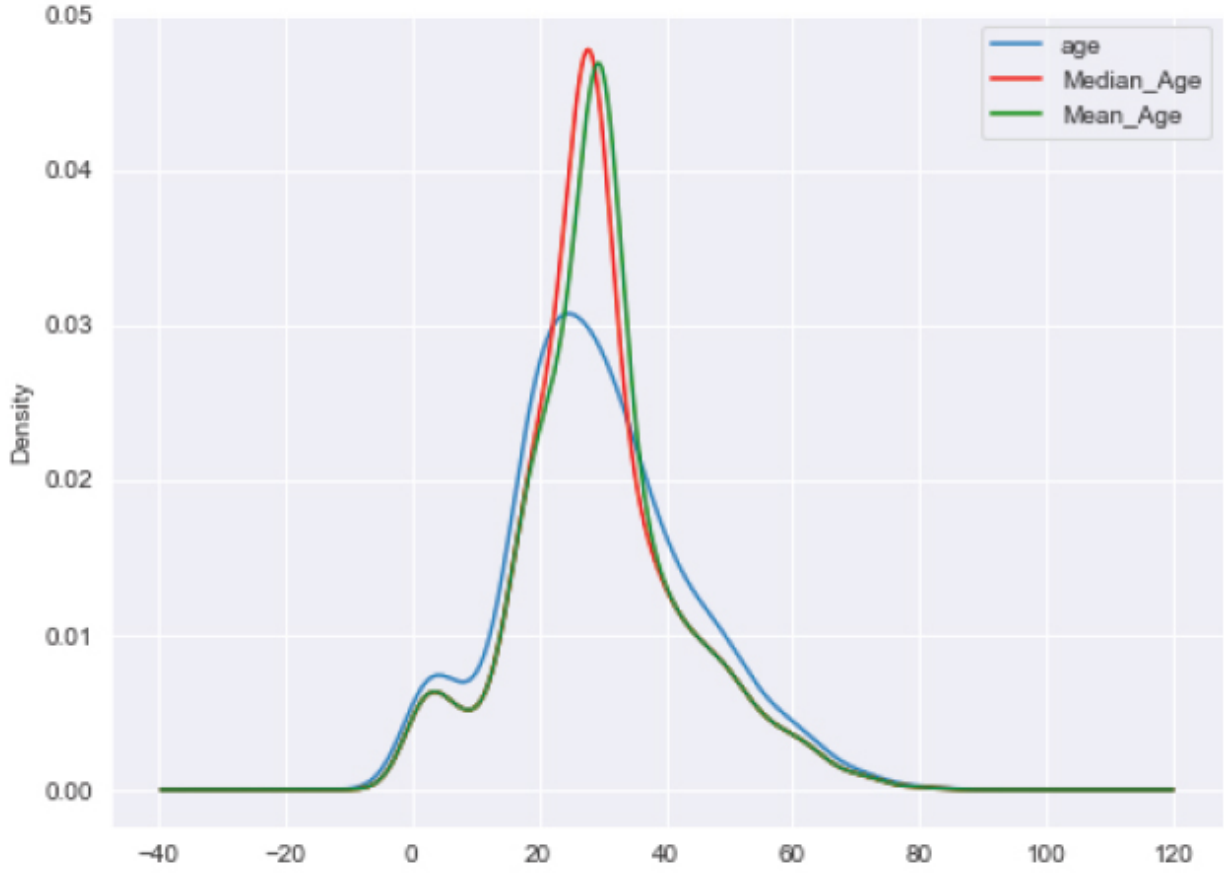
Script 36:

```
fig = plt.figure()
ax = fig.add_subplot( 111 )

titanic_data['age'] .plot(kind='kde', ax=ax)

titanic_data['Median_Age'] .plot(kind='kde', ax=ax, color='red')

titanic_data['Mean_Age'] .plot(kind='kde', ax=ax, color='green')

lines, labels = ax.get_legend_handles_labels()
ax.legend(lines, labels, loc='best')
```

Here is the output of the script above:

Output:

```
<matplotlib.legend.Legend at 0x1b1414a3c70>
```



You can see that the default values in the *age* columns have been distorted by the mean and median imputation, and the overall variance of the dataset has also been decreased.

Recommendations

Mean and Median imputation could be used for the missing numerical data in case the data is missing at random. If the data is normally distributed, mean imputation is better, or else, median imputation is preferred in case of skewed distributions.

2.4.2. Handling Missing Categorical Values

Frequent Category Imputation

One of the most common ways of handling missing values in a categorical column is to replace the missing values with the most frequently occurring values, i.e., the mode of the column. It is for this reason, frequent category imputation is also known as mode imputation. Let's see a real-world example of the frequent category imputation.

We will again use the Titanic dataset. We will first try to find the percentage of missing values in the *age* , *fare*, and *embarked_ town* columns.

Script 37:

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.rcParams["figure.figsize"] = [ 8 , 6 ]
sns.set_style("darkgrid")

titanic_data = sns.load_dataset('titanic')

titanic_data = titanic_data[["embark_town", "age", "fare"]]
titanic_data.head()
titanic_data.isnull().mean()
```

Output:

```
embark_town    0.002245
age    0.198653
fare    0.000000
dtype:  float64
```

The output shows that *embark_town* and *age* columns have missing values. The ratio of missing values for the *embark_ town* column is very less.

Let's plot the bar plot that shows each category in the *embark_town* column against the number of passengers.

Script 38:

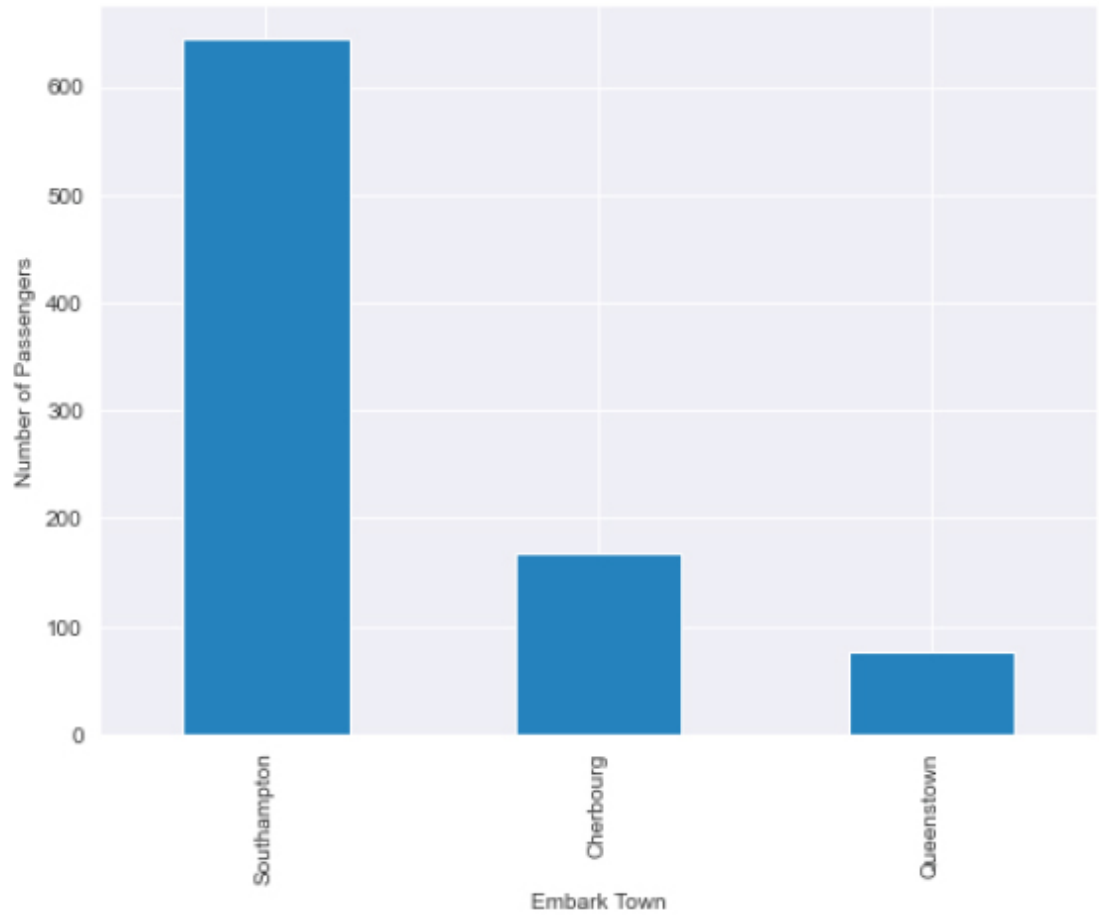
```
titanic_data.embark_town.value_counts().sort_values(ascending=False).plot.bar()
plt.xlabel('Embark Town')
```

```
plt.ylabel('Number of Passengers')
```

The output below clearly shows that most of the passengers embarked from Southampton.

Output:

```
Text(0, 0.5, 'Number of Passengers')
```



Let's make sure if *Southampton* is the mode value for the *embark_town* column.

Script 39:

```
titanic_data.embark_town.mode()
```

Output:

```
0 Southampton
dtype:object
```

Next, we can simply replace the missing values in the *embark town* column by *Southampton* .

Script 40:

```
titanic_data.embark_town.fillna('Southampton', inplace=True)
```

Let's now find the mode of the *age* column and use it to replace the missing values in the *age* column.

Script 41:

```
titanic_data.age.mode()
```

Output:

```
0 24.0
dtype: float64
```

The output shows that the mode of the *age* column is 24. Therefore, we can use this value to replace the missing values in the *age* column.

Script 42:

```
import numpy as np

titanic_data['age_mode'] = titanic_data.age.fillna( 24 )

titanic_data.head( 20 )
```

Output:

	embark_town	age	fare	age_mode
0	Southampton	22.0	7.2500	22.0
1	Cherbourg	38.0	71.2833	38.0
2	Southampton	26.0	7.9250	26.0
3	Southampton	35.0	53.1000	35.0
4	Southampton	35.0	8.0500	35.0
5	Queenstown	NaN	8.4583	24.0
6	Southampton	54.0	51.8625	54.0
7	Southampton	2.0	21.0750	2.0
8	Southampton	27.0	11.1333	27.0
9	Cherbourg	14.0	30.0708	14.0
10	Southampton	4.0	16.7000	4.0
11	Southampton	58.0	26.5500	58.0
12	Southampton	20.0	8.0500	20.0
13	Southampton	39.0	31.2750	39.0
14	Southampton	14.0	7.8542	14.0
15	Southampton	55.0	16.0000	55.0
16	Queenstown	2.0	29.1250	2.0
17	Southampton	NaN	13.0000	24.0
18	Southampton	31.0	18.0000	31.0
19	Cherbourg	NaN	7.2250	24.0

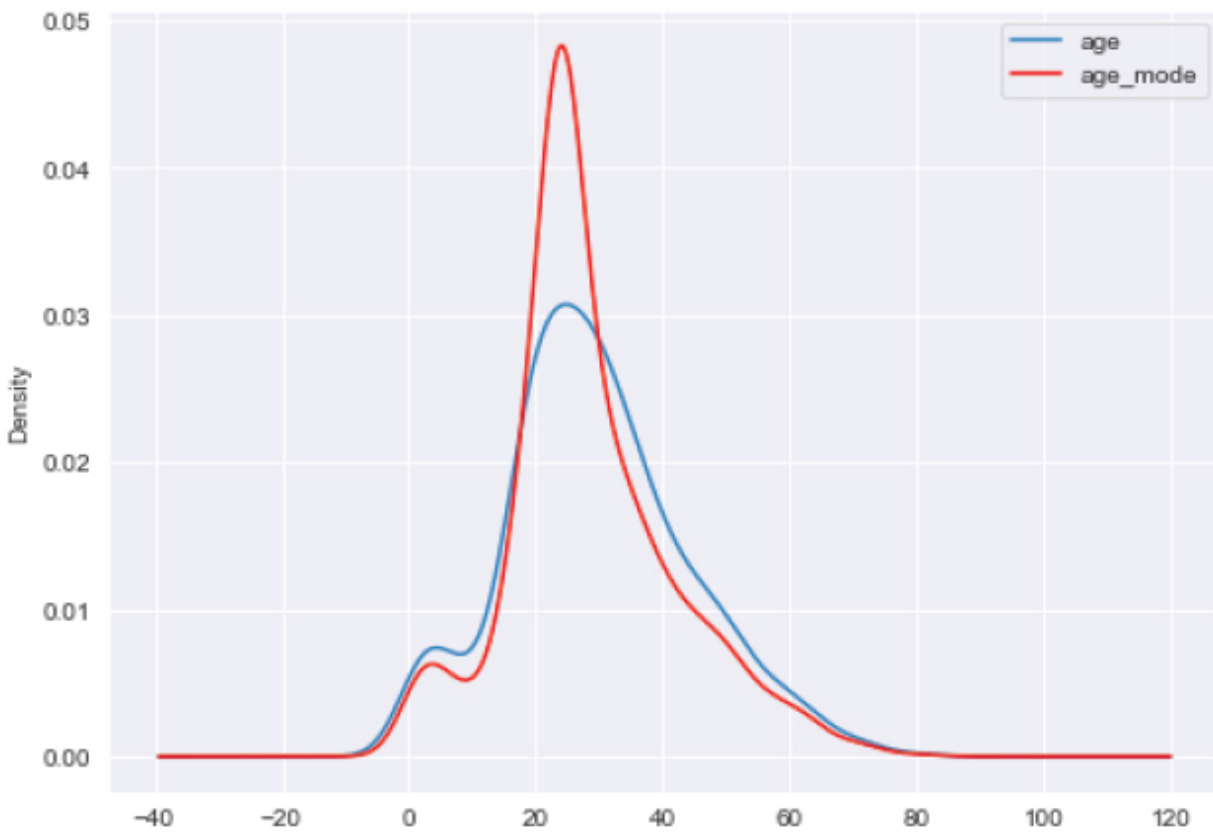
Finally, let's plot the kernel density estimation plot for the original *age* column and the *age* column that contains the mode of the values in place of the missing values.

Script 43:

```
plt.rcParams["figure.figsize"] = [ 8 , 6 ]

fig = plt.figure()
ax = fig.add_subplot( 111 )

titanic_data['age'] .plot(kind='kde', ax=ax)

titanic_data['age_mode'] .plot(kind='kde', ax=ax, color='red')

lines, labels = ax.get_legend_handles_labels()
ax.legend(lines, labels, loc='best')
```

Output:

```
<matplotlib.legend.Legend at 0x1b1416139a0>
```



Missing Category Imputation

Missing value imputation adds an arbitrary category, e.g., *missing* in place of the missing values. Take a look at an example of missing value imputation.

Let's load the Titanic dataset and see if any categorical column contains missing values.

Script 44:

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.rcParams["figure.figsize"] = [ 8 , 6 ]
sns.set_style("darkgrid")

titanic_data = sns.load_dataset('titanic')
titanic_data = titanic_data[["embark_town", "age", "fare"]]
titanic_data.head()
titanic_data.isnull().mean()
```

Output:

```
embark_town 0.002245
age 0.198653
fare 0.000000
dtype: float64
```

The output shows that the *embark_town column* is a categorical column that contains some missing values too. We will apply the missing value imputation to this column.

Script 45:

```
titanic_data.embark_town.fillna('Missing', inplace=True)
```

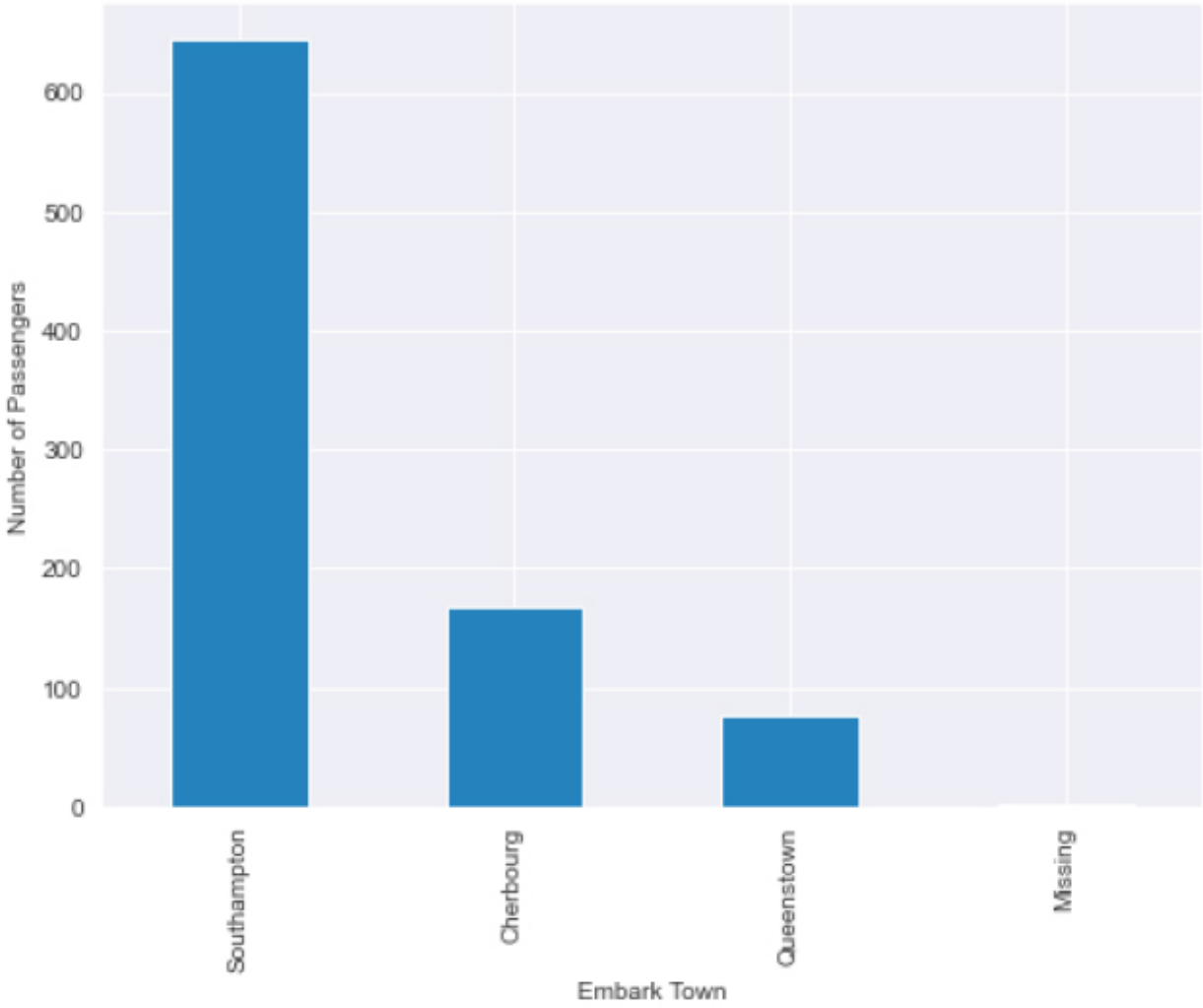
After applying missing value imputation, plot the bar plot for the *embark_town* column. You can see that we have a very small, almost negligible plot for the *missing* column.

Script 46:

```
titanic_data.embark_town.value_counts().sort_values(ascending=False).plot.bar()
plt.xlabel('Embark Town')
plt.ylabel('Number of Passengers')
```

Output:

```
Text(0, 0.5, 'Number of Passengers')
```



Further Readings – Basics of Pandas

1. Check the official documentation here (https://bit.ly/3mQnfOE) to learn more about the Pandas basics.

2. You can learn more about Matplotlib for data plotting at this link (https://matplotlib.org/).

Hands-on Time – Exercises

Now, it is your turn. Follow the instructions in **the exercises below** to check your understanding of Pandas basics that you learned in this chapter. The answers to these questions are given at the end of the book.

Exercise 2.1

Question 1:

What is the major disadvantage of mean and median imputation?

A. Distorts the data distribution

B. Distorts the data variance

C. Distorts the data covariance

D. All of the Above

Question 2:

How do you display the last three rows of a Pandas dataframe named "my_df"?

A. my_df.end(3)

B. my_df.bottom(3)

C. my_df.top(-3)

D. my_df.tail(3)

Question 3:

You can create a Pandas series using a:

A. NumPy Array

B. List

C. Dictionary

D. All of the Above

Exercise 2.2

Replace the missing values in the “deck” column of the Titanic dataset with the most frequently occurring categories in that column. Plot a bar plot for the updated “deck” column. The Titanic dataset can be downloaded using this Seaborn command:

```
import seaborn as sns

sns.load_dataset('titanic')
```